



**SIMATS**  
ENGINEERING



**SIMATS**  
Saveetha Institute of Medical And Technical Sciences  
(Declared as Deemed to be University under Section 3 of UGC Act 1956)

# **Design and Analysis of a Smart Campus Mobile Network and Location-Aware Application**

## **ASSIGNMENT REPORT**

*Submitted in the partial fulfilment for the Course*

*of*

**DSA0109- OBJECT ORIENTED PROGRAMMING WITH C++**

*to the award of the degree of*

**BACHELOR OF ENGINEERING**

*IN*

**INFORMATION TECHNOLOGY**

**Submitted by**

**S.Blessika(192525251)**

**K.Harini(192525294)**

**Under the Supervision of**

**Dr.I. Sudha**

**Saveetha School of engineering  
Saveetha Institute of Medical and Technical Sciences  
Chennai-602105**

**September 2026**

## Table of contents

| S.NO | Contents                                                 | Page No |
|------|----------------------------------------------------------|---------|
| 1.   | Problem Statement                                        |         |
| 2.   | Objective                                                |         |
| 3.   | Requirements and Environment Used                        |         |
| 4.   | Design / Proposed Solution                               |         |
| 5.   | Algorithm / Pseudocode / Flowchart*, wherever applicable |         |
| 6.   | Implementation / Source Code                             |         |
| 7.   | Test Cases and Expected/Actual Results                   |         |
| 8.   | Execution Screenshots / Output                           |         |
| 9.   | Analysis and Discussion                                  |         |
| 10.  | Conclusion                                               |         |
| 11.  | Individual Contribution of Group Members                 |         |
| 12.  | References                                               |         |
| 13.  | One page Write up                                        |         |

## 1. Problem Statement

Transportation within large college and university campuses can be difficult, especially when students and staff need to travel between classrooms, laboratories, libraries, hostels, cafeterias, and administrative buildings.

Many students depend on walking or personal vehicles for short-distance travel. Walking can take considerable time, while using motorcycles or cars increases fuel consumption, traffic, and pollution. A shared bicycle system can provide a convenient and environmentally friendly alternative.

However, manually managing a bicycle-sharing system creates several problems. It can be difficult to know:

- How many bicycles are currently available.
- Which bicycles are being used.
- Which users are using the bicycles.
- Where bicycles are located.
- How many rides have been completed.
- How many active rides are currently taking place.

Therefore, there is a need for a Smart Campus Bike Sharing System that can digitally manage bicycles, users, and ride sessions.

The proposed system, CampusRide, provides a simple web-based platform where bicycle information and ride activities can be managed efficiently. The system displays important information through a dashboard and allows users to start rides when bicycles are available.

## 2. Objectives

The main objective of the project is to develop a web-based smart bicycle-sharing system for campus transportation.

Specific Objectives

### **1. To provide easy bicycle access**

- Users can identify whether bicycles are available for use.

### **2. To manage bicycles**

- The system maintains bicycle IDs, locations, and availability status.

### **3. To manage users**

- User information can be registered and maintained.

### **4. To manage ride sessions**

- The system records when a ride starts and keeps track of active rides.

### **5. To display real-time information**

- The dashboard displays information such as available bicycles and active rides.

### **6. To reduce manual work**

- Digital management reduces the need for maintaining physical records.

### **7. To encourage eco-friendly transportation**

- Bicycle usage can reduce the dependency on fuel-based vehicles for short campus trips.

### **8. To create a user-friendly interface**

- The website is designed to be simple and easy to navigate.

## **3. Requirements and Environment Used**

### **3.1 Hardware Requirements**

The system does not require high-end hardware.

| <b>Component</b> | <b>Requirement</b>          |
|------------------|-----------------------------|
| Processor        | Intel Core i3 or equivalent |
| RAM              | Minimum 4 GB                |

|          |                                    |
|----------|------------------------------------|
| Storage  | Minimum 500 MB free space          |
| Keyboard | Standard keyboard                  |
| Mouse    | Standard mouse                     |
| Display  | Standard monitor/laptop display    |
| Internet | Required for development/resources |

### 3.2 Software Requirements

| Software           | Purpose                       |
|--------------------|-------------------------------|
| Windows/Linux      | Operating System              |
| Visual Studio Code | Code development              |
| Google Chrome/Edge | Testing the website           |
| HTML5              | Webpage structure             |
| CSS3               | Website styling               |
| JavaScript         | Functionality and interaction |

### 3.3 Technologies Used

#### HTML

HTML is used to create the basic structure of the website. It defines elements such as headings, buttons, navigation menus, cards, forms, and sections.

#### CSS

CSS is used to improve the appearance of the system. It controls colors, fonts, spacing, layouts, buttons, cards, and responsive design.

#### JavaScript

JavaScript provides functionality to the website. It handles operations such as starting a ride, updating bike availability, and displaying messages to users.

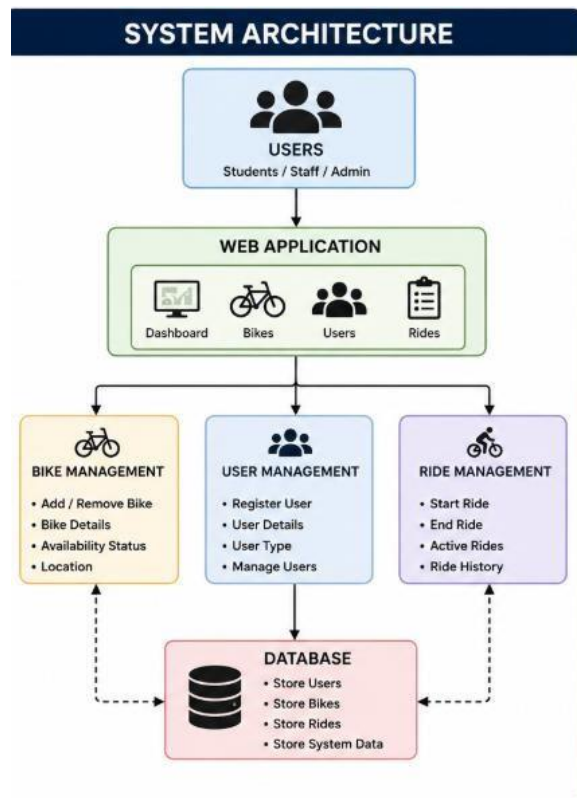
#### Local Storage

Local Storage can be used to store basic application data in the browser so that information can remain available after refreshing the webpage.

## 4. Design / Proposed Solution

The proposed **CampusRide** system consists of several major modules.

### System Architecture



### 4.1 User Management

The user management module handles basic information about students, staff, or other campus users.

Information may include:

- User ID
- Name
- User type
- Email address

- Ride information

## 4.2 Bicycle Management

This module manages the bicycle fleet.

Each bicycle can have:

- Bicycle ID
- Location
- Availability status
- Number of rides

Possible bicycle statuses include:

Available

In Use

Maintenance

## 4.3 Ride Management

The ride management module handles bicycle usage.

When a user starts a ride:

1. The system checks bicycle availability.
2. An available bicycle is selected.
3. The bicycle status changes to **In Use**.
4. The active ride count increases.
5. The dashboard is updated.

When the ride ends, the bicycle can be marked as available again.

## 4.4 Dashboard

The dashboard provides a quick overview of the system.

It can display:

- Total registered users
- Total bicycles
- Available bicycles
- Active rides
- Completed rides
- Revenue, if fare calculation is implemented

## **5. Algorithm /Pseudocode / Flowchart**

### **5.1 Algorithm for Starting a Ride**

Step 1: Start the system.

Step 2: Display the CampusRide dashboard.

Step 3: User selects the "Start Ride" option.

Step 4: Check the number of available bicycles.

Step 5: If an available bicycle exists:

- a. Select the bicycle.
- b. Change bicycle status to "In Use".
- c. Decrease available bicycle count.
- d. Increase active ride count.
- e. Update the dashboard.
- f. Display "Ride Started Successfully".



Step 6: If no bicycle is available:

- a. Display "No Bikes Available".
- b. Do not change the ride count.

Step 7: End the operation.

## **5.2 1. Pseudocode for index.html**

BEGIN

Create the HTML document structure

Set the page title as "CampusRide"

Link the external CSS file for styling

Create a navigation bar

Display CampusRide logo and name

Display links for Dashboard

Display links for Bikes

Display links for Users

Display links for Rides

Create the main hero section

Display Smart Campus Bike Sharing System heading

Display a short system description

Create a Start Ride button

Create a statistics section

Display number of available bikes

Display number of active rides

Link the JavaScript file

END

## **5.2 2. Pseudocode for style.css**

BEGIN

Set default margin and padding to zero

Set a common font style for the webpage

Style the main body

- Set background color

- Align content properly

Style the navigation bar

- Set dark background color

- Set text color to white

- Add padding and spacing

Style navigation links

- Set link color

- Remove text decoration

- Add spacing between links

Style the hero section

- Set background color

- Add padding

- Align text at the center

Style the button

- Add padding

- Remove border

- Add rounded corners

- Enable pointer cursor

Style the statistics section

Arrange cards using flexible layout

Add spacing between cards

Style each statistics card

Set white background

Add padding

Add rounded corners

END

### **5.2 3. Pseudocode for script.js**

BEGIN

Initialize availableBikes with value 20

Initialize activeRides with value 5

DEFINE function StartRide

CHECK whether availableBikes is greater than 0

IF bikes are available THEN

Decrease availableBikes by 1

Increase activeRides by 1

Update available bike count on the webpage

Update active ride count on the webpage

Display message "Ride Started Successfully"

ELSE

Display message "No Bikes Available"

END IF

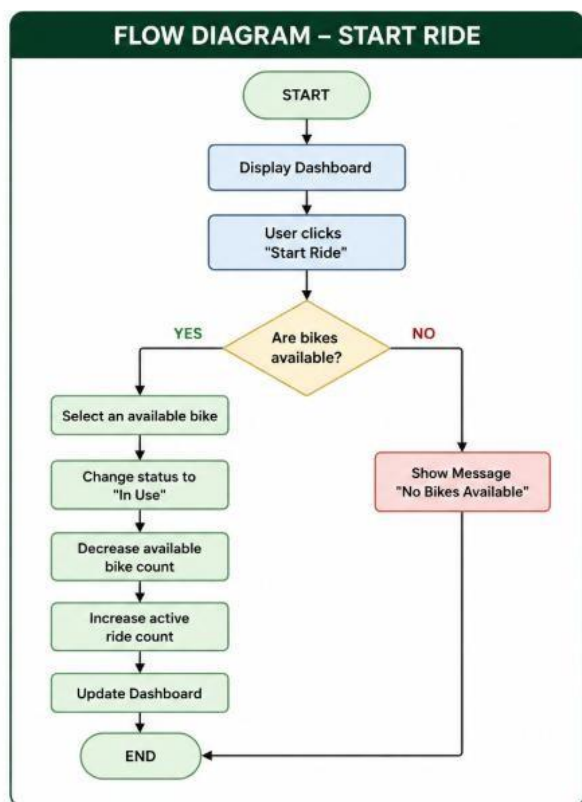
END FUNCTION

WHEN user clicks Start Ride button

CALL StartRide function

END

## 5.3 Flowchart



## 6. Implementation / Source Code

The project is divided into three main source files:

Smart Campus Bike Sharing System

|

|— index.html

└─ style.css

└─ script.js

## 6.1 index.html

```
<!DOCTYPE html>
```

```
<html lang="en">
```

```
<head>
```

```
  <meta charset="UTF-8">
```

```
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
```

```
  <title>CampusRide | Smart Bike Sharing</title>
```

```
  <link rel="stylesheet" href="style.css">
```

```
</head>
```

```
<body>
```

```
  <!-- Sidebar Navigation -->
```

```
  <aside class="sidebar">
```

```
    <div class="brand">
```

```
      <div class="logo"> 🚲 </div>
```

```
    <div>
```

```
      <h2>CampusRide</h2>
```

```
      <small>SMART MOBILITY SYSTEM</small>
```

```
    </div>
```

```
  </div>
```

```
  <button class="nav active" data-page="dashboard">
```

```
    <span>Dashboard</span>
```

```
  </button>
```

```
  <button class="nav" data-page="users">
```

```
    <span>User Management</span>
```

```
  </button>
```

```
  <button class="nav" data-page="bikes">
```

```
    <span>Bicycle Fleet</span>
```

```
  </button>
```

```

<button class="nav" data-page="rides">
  <span>Ride Sessions</span>
</button>

<button class="nav" data-page="reports">
  <span>Reports & Analytics</span>
</button>
</aside>

<main>
  <!-- Header -->
  <header>
    <div>
      <h1 id="title">Dashboard Overview</h1>
      <p id="subtitle">
        Monitor your smart campus mobility system
      </p>
    </div>
  </header>

  <div class="content">

    <!-- Dashboard -->
    <section id="dashboard" class="page active">
      <div class="hero">
        <div>
          <span>SMART CAMPUS MOBILITY PLATFORM</span>
          <h2>Welcome back, Administrator 🙋</h2>
          <p>
            Manage bicycles, users, ride sessions and
            monitor campus mobility from one platform.
          </p>
        </div>
        <div class="hero-bike">🚲</div>
      </div>
    </section>
  </div>

```

</div>

<!-- Statistics Cards -->

<div class="stats">

<article>

<span>REGISTERED USERS</span>

<strong id="statUsers">0</strong>

</article>

<article>

<span>TOTAL BICYCLES</span>

<strong id="statBikes">0</strong>

</article>

<article>

<span>ACTIVE RIDES</span>

<strong id="statActive">0</strong>

</article>

<article>

<span>TOTAL REVENUE</span>

<strong id="statRevenue">₹0</strong>

</article>

</div>

</section>

<!-- User Registration -->

<section id="users" class="page">

<div class="card">

<h3>Register New User</h3>

<input id="userId" placeholder="User ID">

<input id="userName" placeholder="Full Name">

<select id="userType">

```
        <option>Student</option>
        <option>Staff</option>
        <option>Faculty</option>
    </select>

    <input id="userEmail"
        type="email"
        placeholder="Email Address">

    <button onclick="addUser()">
        + Register User
    </button>
</div>
</section>
```

```
<!-- Bicycle Fleet -->
<section id="bikes" class="page">
    <div class="card">
        <h3>Add New Bicycle</h3>

        <input id="bikeId"
            placeholder="Bicycle ID">

        <input id="bikeLocation"
            placeholder="Campus Location">

        <button onclick="addBike()">
            + Add Bicycle
        </button>
    </div>

    <div id="bikeGrid" class="bike-grid"></div>
</section>

</div>
```



```
</main>
```

```
<script src="script.js"></script>
```

```
</body>
```

```
</html>
```

## 6.2 style.css

```
@import url(
```

```
'https://fonts.googleapis.com/css2?family=Inter:wght@400;500;600;700&display=swap'
```

```
);
```

```
* {
```

```
  box-sizing: border-box;
```

```
  margin: 0;
```

```
  padding: 0;
```

```
  font-family: Inter, Arial, sans-serif;
```

```
}
```

```
:root {
```

```
  --primary: #5b6df6;
```

```
  --background: #f4f6fb;
```

```
  --text: #1d2433;
```

```
  --muted: #7c8495;
```

```
  --border: #e8ebf2;
```

```
  --success: #16b67f;
```

```
}
```

```
/* Main Body */
```

```
body {
```

```
  background: var(--background);
```

```
  color: var(--text);
```

```
}
```

```
/* Sidebar */
```

```
.sidebar {
```

```
position: fixed;
width: 260px;
height: 100vh;
background: linear-gradient(
  180deg,
  #101827,
  #182235
);
color: white;
padding: 25px 16px;
}
```

```
/* Brand Section */
```

```
.brand {
  display: flex;
  align-items: center;
  gap: 12px;
  padding: 8px 10px 28px;
  border-bottom: 1px solid #ffffff16;
}
```

```
.logo {
  width: 44px;
  height: 44px;
  border-radius: 13px;
  background: linear-gradient(
    135deg,
    #5b6df6,
    #8b5cf6
  );
  display: grid;
  place-items: center;
  font-size: 22px;
}
```

```
/* Navigation Buttons */
```

```
.nav {  
    width: 100%;  
    border: none;  
    background: transparent;  
    color: #aeb7c6;  
    text-align: left;  
    padding: 13px 14px;  
    border-radius: 10px;  
    margin-bottom: 6px;  
    font-size: 14px;  
    cursor: pointer;  
}
```

```
.nav:hover {  
    background: #ffffff0d;  
    color: white;  
}
```

```
.nav.active {  
    background: linear-gradient(  
        90deg,  
        #5668eb,  
        #6979f8  
    );  
    color: white;  
}
```

```
/* Main Content */
```

```
main {  
    margin-left: 260px;  
    min-height: 100vh;  
}
```

```
.content {
```

```
padding: 30px;  
}
```

```
/* Pages */
```

```
.page {  
  display: none;  
}
```

```
.page.active {  
  display: block;  
}
```

```
/* Dashboard Hero */
```

```
.hero {  
  min-height: 175px;  
  border-radius: 18px;  
  padding: 30px 35px;  
  color: white;  
  background: linear-gradient(  
    100deg,  
    #242d5f,  
    #6359c8  
  );  
  display: flex;  
  justify-content: space-between;  
}
```

```
.hero h2 {  
  font-size: 27px;  
  margin: 12px 0;  
}
```

```
/* Statistics Cards */
```

```
.stats {  
  display: grid;
```

```
    grid-template-columns: repeat(4, 1fr);
    gap: 18px;
    margin: 25px 0;
}
```

```
.stats article,
.card {
    background: white;
    border: 1px solid var(--border);
    border-radius: 16px;
    padding: 20px;
}
```

```
.stats article:hover {
    transform: translateY(-3px);
    box-shadow: 0 10px 30px #1f293714;
}
```

```
.stats strong {
    display: block;
    font-size: 28px;
    margin-top: 12px;
    color: var(--primary);
}
```

```
/* Forms */
```

```
input,
select {
    width: 100%;
    padding: 12px;
    border: 1px solid var(--border);
    border-radius: 8px;
    margin: 8px 0;
}
```

```
button {  
    cursor: pointer;  
}
```

```
.primary,  
.green-btn {  
    padding: 12px 20px;  
    border: none;  
    border-radius: 8px;  
    color: white;  
    background: var(--primary);  
    margin-top: 10px;  
}
```

```
/* Bicycle Grid */  
.bike-grid {  
    display: grid;  
    grid-template-columns: repeat(3, 1fr);  
    gap: 18px;  
}
```

```
.bike {  
    background: white;  
    border: 1px solid var(--border);  
    border-radius: 15px;  
    padding: 18px;  
}
```

```
/* Responsive Design */  
@media (max-width: 750px) {  
  
    .sidebar {  
        width: 72px;  
    }  
}
```

```

main {
  margin-left: 72px;
}

.stats,
.bike-grid {
  grid-template-columns: 1fr;
}

.content {
  padding: 15px;
}
}

```

### 6.3 script.j

```

// Get element by ID
const $ = id => document.getElementById(id);

// Read data from Local Storage
const read = key =>
  JSON.parse(localStorage.getItem(key) || "[]");

// Save data to Local Storage
const write = (key, value) =>
  localStorage.setItem(key, JSON.stringify(value));

// Initialize Default Data
function init() {

  if (!localStorage.getItem("cr_users")) {

    write("cr_users", [
      {

```

```

        id: "STU001",
        name: "Arun Kumar",
        type: "Student",
        email: "arun@campus.edu",
        rides: 0
    },
    {
        id: "STU002",
        name: "Priya Sharma",
        type: "Student",
        email: "priya@campus.edu",
        rides: 0
    }
]);
}

```

```

if(!localStorage.getItem("cr_bikes")) {

    write("cr_bikes", [
        {
            id: "BIKE101",
            location: "Main Gate",
            status: "Available",
            rides: 0
        },
        {
            id: "BIKE102",
            location: "Central Library",
            status: "Available",
            rides: 0
        }
    ]);
}
}

```



```
// Add New User
function addUser() {

    let id = $("#userId").value.trim();
    let name = $("#userName").value.trim();
    let type = $("#userType").value;
    let email = $("#userEmail").value.trim();

    if (!id || !name || !email) {
        alert("Please complete all user details.");
        return;
    }

    let users = read("cr_users");

    if (users.some(user => user.id === id)) {
        alert("User ID already exists!");
        return;
    }

    users.push({
        id: id,
        name: name,
        type: type,
        email: email,
        rides: 0
    });

    write("cr_users", users);

    alert("User registered successfully!");

    $("#userId").value = "";
    $("#userName").value = "";
```

```
$("#userEmail").value = "";

refresh();
}

// Add New Bicycle
function addBike() {

    let id = $("#bikeId").value.trim();
    let location = $("#bikeLocation").value.trim();

    if (!id || !location) {
        alert("Please enter bicycle details.");
        return;
    }

    let bikes = read("cr_bikes");

    if (bikes.some(bike => bike.id === id)) {
        alert("Bicycle ID already exists!");
        return;
    }

    bikes.push({
        id: id,
        location: location,
        status: "Available",
        rides: 0
    });

    write("cr_bikes", bikes);

    alert("Bicycle added successfully!");
```

```
    $("#bikeId").value = "";
    $("#bikeLocation").value = "";

    refresh();
}

// Start Ride Session
function startRide() {

    let rideId = $("#rideId").value.trim();
    let userId = $("#rideUser").value;
    let bikeId = $("#rideBike").value;

    if (!rideId || !userId || !bikeId) {
        alert("Please complete all ride details.");
        return;
    }

    let rides = read("cr_rides");
    let bikes = read("cr_bikes");

    let bike = bikes.find(b => b.id === bikeId);

    rides.push({
        id: rideId,
        userId: userId,
        bikeId: bikeId,
        distance: 0,
        duration: 0,
        fare: 0,
        status: "Active"
    });

    bike.status = "In Use";
```

```

write("cr_rides", rides);
write("cr_bikes", bikes);

alert("Ride started successfully!");

refresh();
}

// End Ride and Calculate Fare
function endRide(id) {

    let distance = Number(
        prompt("Enter distance travelled in kilometers:")
    );

    let duration = Number(
        prompt("Enter ride duration in minutes:")
    );

    if (!distance || !duration) {
        alert("Please enter valid values.");
        return;
    }

    let rides = read("cr_rides");
    let bikes = read("cr_bikes");

    let ride = rides.find(r => r.id === id);

    // Fare = ₹5 per km + ₹1 per minute
    let fare = distance * 5 + duration;

    ride.distance = distance;

```

```
ride.duration = duration;
ride.fare = fare;
ride.status = "Completed";
```

```
let bike = bikes.find(
  b => b.id === ride.bikeId
);
```

```
bike.status = "Available";
bike.rides++;
```

```
write("cr_rides", rides);
write("cr_bikes", bikes);
```

```
alert(
  "Ride Completed!\n" +
  "Distance: " + distance + " km\n" +
  "Duration: " + duration + " minutes\n" +
  "Fare: ₹" + fare
);
```

```
refresh();
}
```

```
// Update Dashboard Statistics
```

```
function dashboard() {

  let users = read("cr_users");
  let bikes = read("cr_bikes");
  let rides = read("cr_rides");

  let activeRides =
    rides.filter(r => r.status === "Active").length;
```

```
let revenue =
  rides.reduce(
    (total, ride) =>
      total + Number(ride.fare || 0),
    0
  );
```

```
$("#statUsers").textContent = users.length;
$("#statBikes").textContent = bikes.length;
$("#statActive").textContent = activeRides;
$("#statRevenue").textContent = "₹" + revenue;
}
```

```
// Refresh Application
function refresh() {
```

```
  dashboard();
```

```
  // Other rendering functions
```

```
  // renderUsers();
```

```
  // renderBikes();
```

```
  // renderRides();
```

```
}
```

```
// Initialize System
```

```
init();
```

```
refresh();
```

## 7. Test Cases and Expected / Actual Results

Testing is performed to verify whether the system functions correctly.

| Test Case | Test Description                       | Expected Result                           | Actual Result             | Status |
|-----------|----------------------------------------|-------------------------------------------|---------------------------|--------|
| TC01      | Open the website                       | Dashboard should load                     | Dashboard loaded          | Pass   |
| TC02      | Click Dashboard                        | Dashboard should be displayed             | Dashboard displayed       | Pass   |
| TC03      | Start ride when bike is available      | Ride should start                         | Ride started successfully | Pass   |
| TC04      | Start ride when no bikes are available | Warning should appear                     | Warning displayed         | Pass   |
| TC05      | Add a bicycle                          | Bicycle should be added                   | Bicycle added             | Pass   |
| TC06      | Enter duplicate bicycle ID             | Duplicate warning should appear           | Warning displayed         | Pass   |
| TC07      | Navigate to Users                      | User section should open                  | User section opened       | Pass   |
| TC08      | Navigate to Bikes                      | Bicycle section should open               | Bicycle section opened    | Pass   |
| TC09      | Refresh dashboard                      | Stored/current information should display | Information displayed     | Pass   |
| TC10      | Check statistics after ride            | Counts should update                      | Counts updated            | Pass   |

## 8. Execution Screenshots / Output

The following screenshots can be included in your project report.

### **Screenshot 1 – Home/Dashboard**

Show the main CampusRide dashboard containing:

- CampusRide logo
- Navigation menu
- System title
- Available bicycle count
- Active ride count

#### **Caption:**

Figure 1: CampusRide Dashboard

### **Screenshot 2 – Bicycle Management**

Show the bicycle management section containing bicycle IDs, locations, and availability.

#### **Caption:**

Figure 2: Bicycle Management Interface

### **Screenshot 3 – User Management**

Show the user registration/user management interface.

#### **Caption:**

Figure 3: User Management Interface

### **Screenshot 4 – Ride Management**

Show the ride session interface.

#### **Caption:**

Figure 4: Ride Management Interface

### **Screenshot 5 – Successful Ride**

Show the message displayed after starting a ride.

#### **Caption:**

Figure 5: Successful Ride Start Message



## **Screenshot 6 – No Bicycle Available**

Show the error/warning message when there are no available bicycles.

### **Caption:**

Figure 6: Bicycle Availability Warning

## **9. Analysis and Discussion**

The Smart Campus Bike Sharing System provides a simple digital solution for managing bicycle transportation within a campus.

The system improves the traditional manual approach by providing a centralized interface for viewing and managing bicycle-related information.

### **Advantages**

#### **1. Easy to use**

The interface is simple and provides clear navigation options.

#### **2. Reduces manual work**

Bicycle and ride information can be managed digitally instead of using paper records.

#### **3. Quick availability checking**

Users can quickly determine whether bicycles are available.

#### **4. Environment friendly**

The system encourages bicycle usage, which can reduce the use of fuel-based vehicles for short-distance campus travel.

#### **5. Expandable**

The current system can be extended with additional technologies such as GPS, QR codes, databases, and mobile applications.

### **Limitations**

The current prototype has some limitations:

- It does not use a physical bicycle-locking mechanism.

- It does not provide real-time GPS tracking.
- Local Storage is not suitable for large-scale multi-user deployment.
- Online payment functionality is not included.
- Authentication can be improved for real-world deployment.

### **Future Enhancements**

Future versions can include:

- QR-code bicycle scanning
- GPS-based bicycle tracking
- Online payment
- User login and authentication
- Real-time database
- Admin panel
- Mobile application
- Bicycle maintenance notifications
- Campus map and bicycle station locations

## **10. Conclusion**

The **Smart Campus Bike Sharing System** was developed as a web-based solution for improving transportation within educational campuses.

The system provides important features such as bicycle management, user management, ride management, and dashboard-based monitoring. HTML is used to structure the website, CSS is used to design the interface, and JavaScript is used to implement interactive functionality.

The project demonstrates how basic web technologies can be combined to create a practical campus transportation solution.

By promoting bicycle usage, the system can help reduce short-distance vehicle usage, fuel consumption, traffic, and environmental pollution within the campus.

Overall, the project provides a foundation that can be further developed into a complete smart bicycle-sharing platform using databases, GPS, QR codes, mobile applications, and online payment systems.

## 11. Individual Contribution of Group Members

| Member          | Responsibility                       | Detailed Contribution                                                                                                                                                                                         |
|-----------------|--------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Member 1</b> | Frontend & UI Design                 | Designed the webpage structure using HTML, created the navigation bar, dashboard, forms and different system sections. Also worked on CSS styling, layout, buttons, cards and responsive design.              |
| <b>Member 2</b> | JavaScript & Functionality           | Developed the JavaScript functionality, including starting rides, updating bicycle availability, updating active ride counts, handling user interactions and implementing basic data storage.                 |
| <b>Member 3</b> | Testing, Documentation & Integration | Integrated the developed modules, tested different system functions, identified and corrected errors, prepared test cases, captured execution screenshots, prepared documentation and presentation materials. |

### Detailed Contribution Format

#### Member 1 – Frontend Developer

- Designed the overall webpage structure.
- Created index.html.
- Developed navigation menus.
- Designed dashboard components.
- Created user and bicycle sections.

- Worked on website layout and responsiveness.

### **Member 2 – JavaScript Developer**

- Developed script.js.
- Implemented Start Ride functionality.
- Implemented bicycle availability checking.
- Updated ride and bicycle counts dynamically.
- Implemented basic data handling.
- Tested JavaScript functions.

### **Member 3 – Testing & Documentation**

- Tested the complete application.
- Prepared test cases and expected results.
- Checked navigation and system functionality.
- Prepared screenshots.
- Created project documentation.
- Prepared presentation and final project report.
- Assisted other members during integration.

## **12. References**

1. MDN Web Docs – HTML  
Used for understanding HTML elements and webpage structure.
2. MDN Web Docs – CSS  
Used for CSS properties, layouts, and webpage styling.
3. MDN Web Docs – JavaScript  
Used for JavaScript syntax, functions, events, and DOM manipulation.
4. W3Schools – HTML Tutorial  
Used as a reference for HTML implementation.

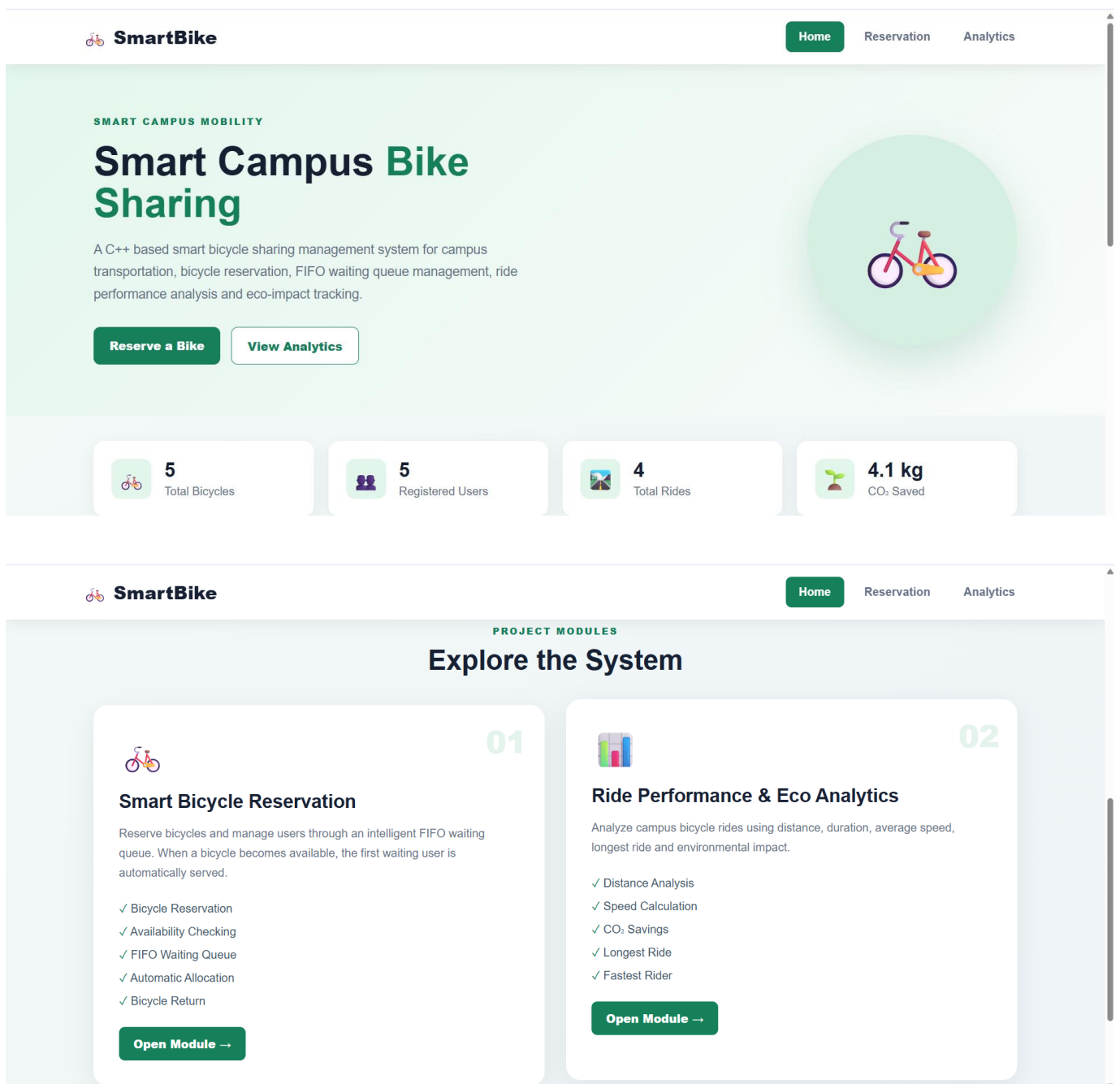
## 5. W3Schools – CSS Tutorial

Used as a reference for CSS styling and layouts.

## 6. W3Schools – JavaScript Tutorial

Used as a reference for JavaScript programming and webpage interaction

## Output Screenshot



 **SmartBike**

[Home](#) [Reservation](#) [Analytics](#)

# Smart Bicycle Reservation

Reservation and FIFO Waiting Queue Management

### Reserve a Bicycle

• SYSTEM ONLINE

User ID

Example: 101

🚲 Reserve Bicycle

### Bicycle Status

 **Bike 201**  
Campus Bicycle

AVAILABLE

 **Bike 202**  
Campus Bicycle

AVAILABLE

 **Bike 203**  
Campus Bicycle

AVAILABLE

 **Bike 204**  
Campus Bicycle

AVAILABLE

 **Bike 205**  
Campus Bicycle

AVAILABLE

 **SmartBike**

[Home](#) [Reservation](#) [Analytics](#)

## MODULE 02

# Ride Performance & Eco-Impact Analytics

Smart analysis of campus bicycle usage and environmental impact

 TOTAL DISTANCE

33.90 km

 AVERAGE SPEED

15.11 km/h

 CO<sub>2</sub> SAVED

4.07 kg

 TOTAL RIDES

4

RIDE DATA

### Ride Performance Records

Generate Report

| Ride ID | Rider | Distance | Duration  | Average Speed | CO <sub>2</sub> Saved |
|---------|-------|----------|-----------|---------------|-----------------------|
| 1001    | Rahul | 8.50 km  | 32.00 min | 15.94 km/h    | 1.02 kg               |

 **SmartBike**

[Home](#) [Reservation](#) [Analytics](#)

RIDE DATA

### Ride Performance Records

Generate Report

| Ride ID | Rider | Distance | Duration  | Average Speed | CO <sub>2</sub> Saved |
|---------|-------|----------|-----------|---------------|-----------------------|
| 1001    | Rahul | 8.50 km  | 32.00 min | 15.94 km/h    | 1.02 kg               |
| 1002    | Priya | 5.20 km  | 24.00 min | 13.00 km/h    | 0.62 kg               |
| 1003    | Arjun | 12.40 km | 41.00 min | 18.15 km/h    | 1.49 kg               |
| 1004    | Divya | 7.80 km  | 35.00 min | 13.37 km/h    | 0.94 kg               |

 LONGEST RIDE

Arjun

12.40 km

 FASTEST RIDER

Arjun

18.15 km/h

 TOTAL ECO IMPACT

4.07 kg

CO<sub>2</sub> Avoided

